

COMPOSITE APLICADO EN ESTRUCTURAS ESPACIALES

Patrones de Diseno: *Composite*

Jerarquia y desplazamiento de componentes
mediante recursividad

Autor: **Angie Nathaly Cisneros Guerrrero**

20242020084

Fecha: Septiembre 2026

Contents

1	Diagrama UML	2
2	Explicacion de las Clases	3
2.1	Interfaz Component - Estructura	3
2.1.1	Clase Leaf - Mueble	3
2.2	Patron Composite - Compuesto	5
3	Codigo de Cliente	7
3.1	Clase Main	7
4	Pruebas y Ejecucion	9
4.1	Prueba de Ejecucion en Consola	9
5	Repositorio	10
5.1	Enlace GitHub	10
5.2	Estructura de Paquetes	10

Diagrama UML

Descripcion General

El proyecto implementa el patron de diseno estructural **Composite** para modelar objetos en una estructura de arbol que representa una jerarquia de "parte-todo". En este diseno, la clase cliente puede tratar de manera uniforme objetos individuales (Muebles) y agrupaciones de estos objetos (Pisos y Casas Prefabricadas) compartiendo una misma interfaz de desplazamiento y visualizacion.

Diagrama de Clases

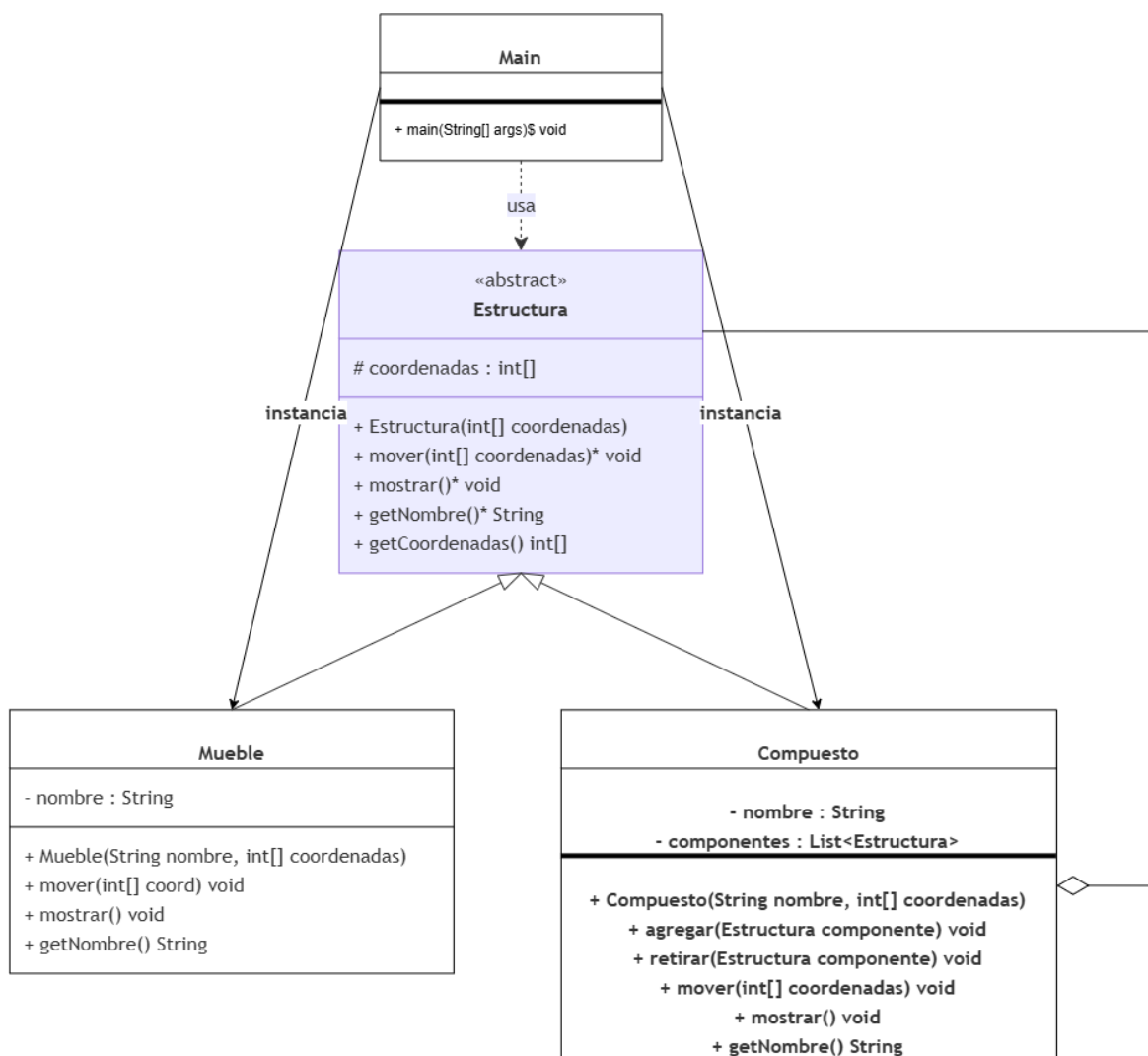


Figure 1: Diagrama de Clases.

Explicacion de las Clases

Interfaz Component - Estructura

Descripcion

Clase abstracta base. Declara los metodos comunes para todas las entidades en el sistema. Gestiona el estado de las coordenadas en el espacio 2D.

Codigo Fuente

```
1 package casa.composite;
2
3 public abstract class Estructura {
4     protected int[] coordenadas = new int [2];
5
6     public Estructura(int[] coordenadas) {
7         this.coordenadas[0]=coordenadas[0];
8         this.coordenadas[1]=coordenadas[1];
9     }
10
11     public abstract void mover(int[] coordenadas);
12     public abstract void mostrar();
13     public abstract String getNombre();
14
15     //Getters
16     public int[] getCoordenadas() {
17         return coordenadas;
18     }
19 }
```

2.1.1 Clase Leaf - Mueble

Descripcion

Representa las "hojas" del arbol jerarquico. No tienen elementos subyacentes. Ejecutan la accion de movimiento actualizando sus propias coordenadas.

Codigo Fuente

```
1 package casa.composite;
2
3 public class Mueble extends Estructura{
4
5     private String nombre;
6
7     public Mueble(String nombre, int[] coordenadas) {
8         super(coordenadas);
9         this.nombre = nombre;
10    }
11
12    @Override
13    public void mover(int[] coord) {
```

```
14     coordenadas[0]=coord[0];
15     coordenadas[1]=coord[1];
16     System.out.println("_->_Moviendo_mueble_" + nombre + "'_a_"
17         + "nuevas_coordenadas_" + coord[0] + ",_" + coord[1] + ")");
18
19     @Override
20     public void mostrar() {
21         System.out.println("_Mueble:_" + nombre + "_en_" +
22             coordenadas[0] + ",_" + coordenadas[1] + ")");
23     }
24
25     @Override
26     public String getNombre() {
27         return nombre;
28     }
29 }
```

Patron Composite - Compuesto

Descripcion

Actua como el contenedor de las hojas o de otros contenedores. Mantiene una coleccion de objetos Estructura. Al recibir una solicitud, este objeto delega recursivamente el trabajo a los elementos secundarios y ejecuta su propio comportamiento.

Codigo Fuente

```
1 package casa.composite;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Compuesto extends Estructura{
7
8     private String nombre;
9     private List<Estructura> componentes = new ArrayList<>();
10
11     public Compuesto(String nombre, int[] coordnedas) {
12         super(coordnedas);
13         this.nombre = nombre;
14     }
15
16     public void agregar(Estructura componente) {
17         componentes.add(componente);
18         System.out.println("Agregando el: " + componente.getNombre
19             () + " al: " + this.nombre + " en (" + coordenadas[0] + ", " +
20             coordenadas[1] + ")");
21         componente.mover(this.coordenadas);
22     }
23
24     public void retirar(Estructura componente) {
25         componentes.remove(componente);
26     }
27
28     @Override
29     public void mover(int[] coordenadas) {
30         System.out.println("\nMoviendo el: " + nombre + " al (" +
31             coordenadas[0] + ", " + coordenadas[1] + ")");
32         this.coordenadas[0]=coordenadas[0];
33         this.coordenadas[1]=coordenadas[1];
34         for (Estructura componente : componentes){
35             componente.mover(coordenadas);
36         }
37     }
38
39     @Override
40     public void mostrar() {
41         System.out.println("\n la estructura: " + nombre+ " esta en ("
42             + coordenadas[0] + ", " + coordenadas[1] + ")");
43         for (Estructura componente : componentes) {
44             componente.mostrar();
45         }
46     }
47 }
```

```
41     }
42 }
43
44 @Override
45 public String getNombre() {
46     return nombre;
47 }
48 }
```

Explicacion

- **Gestion de Hijos:** Utiliza un `ArrayList<Estructura>` para agrupar nodos hoja (Mueble) o ramas secundarias (Compuesto).
- **Delegacion de movimiento (**mover**):** Actualiza sus propias coordenadas y posteriormente itera sobre su lista de hijos aplicando `componente.mover()`.
- **Sincronizacion en **agregar()**:** Fuerza al nuevo componente a igualar sus coordenadas a las del nodo contenedor padre para mantener la consistencia del arbol.

Codigo de Cliente

Clase Main

Codigo Fuente

```
1 package casa.composite;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Estructura sofa = new Mueble("Sofa_de_Cuero", new int[]{0,0})
8             ;
9         Estructura tv = new Mueble("Televisor_55", new int[]{2,0});
10        Estructura cama = new Mueble("Cama_Doble", new int[]{0,5});
11        Estructura armario = new Mueble("Armario_Ropa", new int
12            []{3,5});
13
14        Compuesto piso1 = new Compuesto("Piso_1", new int[]{0,0});
15        piso1.agregar(sofa);
16        piso1.agregar(tv);
17
18        Compuesto piso2 = new Compuesto("Piso_2", new int[] {0, 0});
19        piso2.agregar(cama);
20        piso2.agregar(armario);
21
22        Compuesto casaCompleta = new Compuesto("Casa_Prefabricada",
23            new int[]{10, 10});
24        casaCompleta.agregar(piso1);
25        casaCompleta.agregar(piso2);
26
27        System.out.println("---_ESTRUCTURA_INICIAL_---");
28        casaCompleta.mostrar();
29
30        System.out.println("\n---_INICIANDO_TRASLADO_DE_LA_CASA_---")
31            ;
32        casaCompleta.mover(new int[]{50, 20});
33
34        System.out.println("\n---_ESTRUCTURA_FINAL_---");
35        casaCompleta.mostrar();
36    }
37 }
```

Flujo de Ejecucion

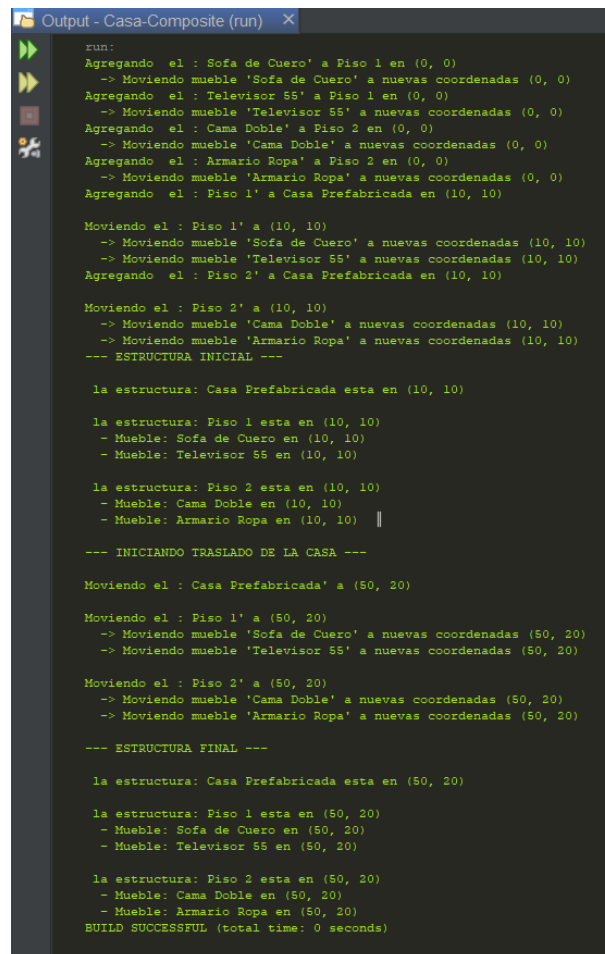
1. Se inicializan hojas (Mueble) con coordenadas relativas previas a ser englobadas.
2. Se agrupan los muebles dentro de contenedores intermedios (Piso 1 y Piso 2). Durante la insercion, las coordenadas de los muebles se actualizan automaticamente.
3. Se ensambla la jerarquia final acoplando los pisos dentro de un componente raiz (Casa Prefabricada).

4. Se emite la directiva unificada `mover()` al elemento raiz. La operacion desciende en cascada trasladando los pisos y posteriormente los muebles contenidos en ellos a la nueva ubicacion espacial.

Pruebas y Ejecucion

Prueba de Ejecucion en Consola

Prueba



```
run:
Agregando el : Sofa de Cuero' a Piso 1 en (0, 0)
-> Moviendo mueble 'Sofa de Cuero' a nuevas coordenadas (0, 0)
Agregando el : Televisor 55' a Piso 1 en (0, 0)
-> Moviendo mueble 'Televisor 55' a nuevas coordenadas (0, 0)
Agregando el : Cama Doble' a Piso 2 en (0, 0)
-> Moviendo mueble 'Cama Doble' a nuevas coordenadas (0, 0)
Agregando el : Armario Ropa' a Piso 2 en (0, 0)
-> Moviendo mueble 'Armario Ropa' a nuevas coordenadas (0, 0)
Agregando el : Piso 1' a Casa Prefabricada en (10, 10)

Moviendo el : Piso 1' a (10, 10)
-> Moviendo mueble 'Sofa de Cuero' a nuevas coordenadas (10, 10)
-> Moviendo mueble 'Televisor 55' a nuevas coordenadas (10, 10)
Agregando el : Piso 2' a Casa Prefabricada en (10, 10)

Moviendo el : Piso 2' a (10, 10)
-> Moviendo mueble 'Cama Doble' a nuevas coordenadas (10, 10)
-> Moviendo mueble 'Armario Ropa' a nuevas coordenadas (10, 10)
--- ESTRUCTURA INICIAL ---

la estructura: Casa Prefabricada esta en (10, 10)

la estructura: Piso 1 esta en (10, 10)
- Mueble: Sofa de Cuero en (10, 10)
- Mueble: Televisor 55 en (10, 10)

la estructura: Piso 2 esta en (10, 10)
- Mueble: Cama Doble en (10, 10)
- Mueble: Armario Ropa en (10, 10)

--- INICIANDO TRASLADO DE LA CASA ---

Moviendo el : Casa Prefabricada' a (50, 20)

Moviendo el : Piso 1' a (50, 20)
-> Moviendo mueble 'Sofa de Cuero' a nuevas coordenadas (50, 20)
-> Moviendo mueble 'Televisor 55' a nuevas coordenadas (50, 20)

Moviendo el : Piso 2' a (50, 20)
-> Moviendo mueble 'Cama Doble' a nuevas coordenadas (50, 20)
-> Moviendo mueble 'Armario Ropa' a nuevas coordenadas (50, 20)

--- ESTRUCTURA FINAL ---

la estructura: Casa Prefabricada esta en (50, 20)

la estructura: Piso 1 esta en (50, 20)
- Mueble: Sofa de Cuero en (50, 20)
- Mueble: Televisor 55 en (50, 20)

la estructura: Piso 2 esta en (50, 20)
- Mueble: Cama Doble en (50, 20)
- Mueble: Armario Ropa en (50, 20)
BUILD SUCCESSFUL (total time: 0 seconds)
```

Figure 2: Salida de la ejecucion mostrando la actualizacion en cascada de las coordenadas

Repositorio

Enlace GitHub

- <https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Estructura/Casa-Composite>

Estructura de Paquetes

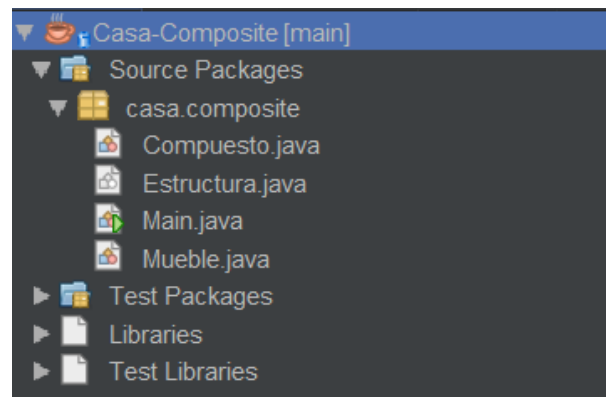


Figure 3: Arbol del proyecto en el IDE.